Eserciziario
Progettazione di Software Sicuro - Laboratorio

Valentina Ciriani, Elvinia Riccobene

Laurea Triennale in Sicurezza dei Sistemi e delle Reti Informatiche

Gioco dell'otto

Java [pt. 2.5]

Il gioco dell'otto[1] consiste di una griglia quadrata, divisa in 3 righe e 3 colonne, su cui sono posizionate in modo casuale 8 tessere, numerate a partire da 1. Le tessere possono scorrere in orizzontale o verticale, ma il loro spostamento è limitato dall'esistenza di un singolo spazio vuoto. Scopo del gioco è riordinare le tessere: la tessera con il numero 1 in alto a sinistra, e quelle con gli altri numeri a seguire in modo ordinato da sinistra a destra e dall'alto in basso, con la cella vuota alla fine, come mostrato in Tabella 1.

1	**2**	**3**
4	**5**	**6**
7	**8**	0

Table 1: Configurazione finale

Scrivere una classe *GiocoDello8* che rappresenti una griglia 3×3; tale classe deve permettere di muovere le tessere nella griglia. Le tessere sono rappresentate da numeri nell'intervallo $[1, 8]$ e la cella vuota da 0.

All'inizio la griglia deve contenere le tessere nell'ordine mostrato in Tabella 2.

2	**3**	**6**
1	**5**	0
4	**7**	**8**

Table 2: Configurazione iniziale

Metodo *move* La classe deve disporre di un metodo *move(int i, int j, int h, int k)* che permetta di spostare in posizione (h, k) la tessera in posizione (i, j). Non bisogna fare alcun controllo sul fatto che i, j, h, k siano indici validi e che la mossa sia fattibile.

Metodo *isSolved* La classe deve disporre di un metodo booleano *isSolved()* che dica se il rompicapo è stato risolto, cioè se i numeri sono disposti in ordine crescente (a partire da 1) da sinistra a destra e dall'alto in basso, e se l'ultima cella è vuota (come in tabella 9).

JML

Scrivere in JML la seguente postcondizione al costruttore:

[1] Il gioco dell'otto è una versione modificata del gioco del 15 (`http://en.wikipedia.org/wiki/Fifteen_puzzle`).

- per ogni tessera numerata con un numero dell'intervallo [1,8] c'è una cella che la contiene; inoltre c'è una cella vuota (nota che la cella vuota è identificata con 0). **[pt. 1.5]**

Scrivere in JML le seguenti precondizioni al metodo *move*:

- gli indici i, j, h e k sono indici validi della griglia; **[pt. 0.5]**
- la cella identificata da (i, j) contiene una tessera; **[pt. 0.5]**
- la cella identificata da (h, k) è la cella vuota; **[pt. 0.5]**
- le celle identificate dagli indici (i, j) e (h, k) sono contigue. **[pt. 1]**

Junit

- In JUnit, per il metodo *move*, scrivere:
  - un caso di test in cui si mostri che lo spostamento di una tessera da (i, j) ad (h, k) (scelti da voi in modo che identifichino celle contigue, che (i, j) contenga una tessera, e che (h, k) sia vuota) avviene in modo corretto; **[pt. 1]**
  - un caso di test in cui si mostri che lo spostamento di una tessera da una cella piena (i, j) a una cella piena (h, k) viene eseguito portando la griglia in una configurazione errata (sparisce la tessera presente nella cella di destinazione). Il caso di test si aspetta che, se si prova a spostare una tessera in una cella piena, lo spostamento non dovrebbe essere eseguito: invece il metodo esegue lo spostamento. **[pt. 1]**
- scrivere in JUnit un caso di test che evidenzi che la griglia impostata dal costruttore non contiene la soluzione del rompicapo; **[pt. 1]**
- scrivere in JUnit un caso di test che evidenzi che è possibile eseguire una serie di 7 mosse che portano alla soluzione del rompicapo; nella Tabella 3 sono mostrati i parametri che bisogna usare nelle 7 chiamate a *move*. **[pt. 1]**

i	**j**	**h**	**k**
0	2	1	2
0	1	0	2
0	0	0	1
1	0	0	0
2	0	1	0
2	1	2	0
2	2	2	1

Table 3: Mosse per ottenere la soluzione del rompicapo a partire dalla configurazione impostata dal costruttore

Knight's tour

Java [pt. 2.5]

Nel gioco degli scacchi, il cavallo può muoversi solo eseguendo un movimento ad **L**, cioè muovendosi prima in orizzontale (a sinistra o destra) di 1/2 celle e poi in verticale (verso l'alto o verso il basso) di 2/1 celle. Nella Tabella 4 sono mostrate le mosse che il cavallo posizionato in `c2` può fare in una scacchiera 5×5: può andare in `b4`, `d4`, `a3`, `e3`, `a1`, `e1`.

	a	b	c	d	e
5					
4		×		×	
3	×				×
2			♞		
1	×				×

Table 4: Possibili mosse di un cavallo posizionato in `c2`

Il problema *Knight's tour* prevede di far eseguire al cavallo un *tour* completo della scacchiera. Il cavallo, all'inizio, deve essere posizionato in una cella della scacchiera. Successivamente, bisogna far eseguire una serie di mosse al cavallo, seguendo le seguenti regole:

- il cavallo deve essere mosso seguendo le regole degli scacchi (movimento a **L**);
- quando il cavallo visita una cella, questa deve essere marchiata come *visitata*;
- una cella non può essere visitata due volte.

Il problema prevede che il cavallo esegua un tour *completo*, cioè che visiti tutte le celle della scacchiera. Il cavallo deve eseguire un *open tour*, cioè la cella iniziale e la cella finale del tour devono essere diverse.

Scrivere una classe *KnightTour* che rappresenti una scacchiera 5×5 di booleani: la scacchiera deve memorizzare se una cella è già stata visitata o meno. La classe deve permettere anche di memorizzare le coordinate in cui si trova attualmente il cavallo. Inoltre la classe deve disporre di un metodo che permetta di muovere il cavallo nella scacchiera.

All'inizio le coordinate del cavallo sono $(0, 0)$ e la scacchiera indica che solo la cella $(0, 0)$ è stata visitata.

Metodo *move* La classe deve disporre di un metodo *move(int h, int k)* che permetta di spostare in posizione (h, k) il cavallo e che memorizzi che la cella (h, k) della scacchiera è stata visitata. Non bisogna fare alcun controllo sul fatto che h, k siano indici validi e che la mossa sia corretta (rispetti le regole della mossa del cavallo).

Metodo *isTourCompleted* La classe deve disporre di un metodo booleano *isTourCompleted()* che dica se il cavallo ha eseguito un tour completo della scacchiera.

JML

Scrivere in JML le seguenti postcondizioni al costruttore:

- la cella $(0, 0)$ è marcata come *visitata*; **[pt. 0.25]**
- tutte le celle della scacchiera tranne $(0, 0)$ sono marcate come *non visitate*. **[pt. 1.25]**

Scrivere in JML le seguenti precondizioni al metodo *move(int h, int k)*:

- gli indici h e k sono indici validi della scacchiera; **[pt. 0.5]**
- la cella identificata da (h, k) non è già stata visitata; **[pt. 0.5]**
- la mossa dalla posizione corrente del cavallo ad (h, k) è una mossa ad **L** corretta. **[pt. 1.5]**

Junit

- In JUnit, per il metodo *move*, scrivere:
  - un caso di test in cui si mostri che una mossa valida del cavallo (una mossa ad **L** corretta e tale per cui (h, k) identifichino una cella non ancora visitata) avviene in modo corretto: il cavallo viene spostato in (h, k) e la cella (h, k) viene marcata come *visitata*; **[pt. 1]**
  - un caso di test in cui si mostri che una mossa non valida (non è una mossa ad **L**) viene eseguita. Il caso di test si aspetta che, se si prova ad eseguire una mossa non valida, la mossa non dovrebbe essere eseguita: invece il metodo esegue la mossa. **[pt. 1]**
- scrivere in JUnit un caso di test che evidenzi che sulla scacchiera iniziale (quella costruita dal costruttore) non è stato eseguito un tour completo; **[pt. 1]**
- scrivere in JUnit un caso di test che evidenzi che è possibile eseguire una serie di 24 mosse che portano all'esecuzione di un tour completo; nella Tabella 5 sono mostrati i parametri che bisogna usare nelle 24 chiamate a *move*. **[pt. 1]**

h	2	4	3	4	2	0	1	2	4	3	1	0	1	3	4	2	0	1	3	4	3	2	0	1
k	1	0	2	4	3	4	2	4	3	1	0	2	4	3	1	0	1	3	4	2	0	2	3	1

Table 5: Mosse per ottenere un tour completo a partire dalla configurazione impostata dal costruttore

Sokoban

Java [pt. 2.5]

Sokoban è un gioco in cui un giocatore deve spostare delle casse su una scacchiera. Il giocatore può spostarsi solo nelle quattro direzioni, di un passo alla volta. Se sul suo cammino c'è una cassa, anche la cassa viene spostata nella direzione in cui si sta muovendo.

Consideriamo una versione semplificata del gioco in cui c'è solo una cassa e non ci sono ostacoli sulla scacchiera (tranne le quattro pareti).

Scrivere una classe *Sokoban* che permetta di giocare a Sokoban su una scacchiera di dimensione 5×5. Modellare la scacchiera con una matrice di booleani in cui una cella è *true* se contiene la cassa, *false* altrimenti. La posizione del giocatore sulla scacchiera, invece, deve essere modellata tramite due variabili intere che identifichino le coordinate di riga e colonna.

Il costruttore della classe deve inizializzare la scacchiera come mostrato in Tabella 6.

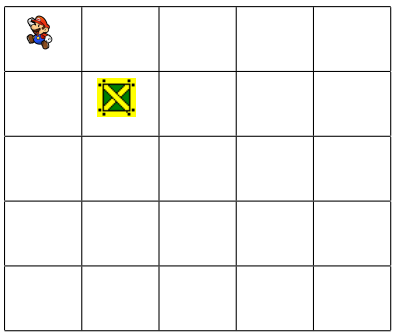

Table 6: Configurazione iniziale del Sokoban

Metodo *move* La classe deve disporre di un metodo booleano *move(int dir)* che permetta di muovere il giocatore sulla scacchiera. Il parametro *dir* indica la direzione che deve essere presa:

- 1: verso l'alto;
- 2: verso destra;
- 3: verso il basso;
- 4: verso sinistra.

A seconda della posizione del giocatore e della cassa, il giocatore viene mosso o meno, e la cassa viene spostata o meno. Il metodo ritorna *true* se il giocatore si è spostato (con o senza la cassa), *false* altrimenti. Vediamo la situazione in cui viene richiesto di muovere il giocatore verso l'alto (parametro *dir* uguale a 1):

- se il giocatore è sulla prima riga, la mossa non deve essere eseguita;
- se il giocatore non è sulla prima riga e non c'è la cassa nella cella sopra di lui, la mossa viene eseguita;
- se il giocatore è sulla seconda riga e c'è la cassa sopra di lui, la mossa non deve essere eseguita;
- se il giocatore è su una riga successiva alla seconda e c'è la cassa sopra di lui, la mossa viene eseguita; in questo caso anche la cassa viene spostata di conseguenza.

I movimenti nelle altre tre direzioni sono simili.

JML

Scrivere in JML la seguente postcondizione al costruttore:

- tutte le celle della scacchiera tranne la cella (1, 1) valgono *false*. [**pt. 1.25**]

Scrivere in JML la seguente precondizione al metodo *move(int dir)*:

- il valore di *dir* è compreso tra 1 e 4. [**pt. 0.25**]

Scrivere in JML le seguenti postcondizioni al metodo *move(int dir)*:

- il numero di celle della scacchiera che valgono *true* è 1; [**pt. 1.25**]
- almeno una delle due coordinate della posizione del giocatore rimane invariata tra prima e dopo l'esecuzione del metodo. [**pt. 1.25**]

Junit

In JUnit, scrivere i seguenti casi di test nella classe *SokobanTest*:

- un caso di test che mostri che, se il giocatore si trova di fronte alla parete nord, non può muoversi verso l'alto (basarsi sul valore ritornato dal metodo); [**pt. 1**]
- un caso di test che mostri una situazione in cui il giocatore riesce a spostare la cassa (basarsi sul valore ritornato dal metodo); [**pt. 1**]
- un caso di test che mostri una situazione in cui il giocatore non riesce a spostare la cassa (basarsi sul valore ritornato dal metodo); [**pt. 1**]
- un caso di test che mostri che il giocatore può spostare la cassa in basso a destra (in posizione (4, 4)); rendere pure pubblici i campi della classe per eseguire il test. [**pt. 1**]

TicTacToe

Java [pt. 2]

Il gioco del *tris* si gioca su una griglia quadrata di 3×3 celle. I giocatori, a turno, disegnano il proprio simbolo in una cella vuota (un giocatore ha come simbolo **X** e l'altro **O**). Vince il giocatore che riesce a disporre il proprio simbolo in linea retta orizzontale, verticale o diagonale. Se la griglia viene riempita senza che nessuno dei giocatori sia riuscito a completare una linea retta di tre simboli, il gioco finisce in parità.

X		**O**
0	**X**	
		X

Table 7: Esempio di griglia in cui ha vinto **X**

Scrivere una classe *TicTacToe* che rappresenti una scacchiera 3×3 di interi; un cella vale:

- 0 se non contiene alcun segno;
- 1 se contiene una **X**;
- 2 se contiene uno **O**.

All'inizio la griglia è vuota.

Metodo *set* La classe deve disporre di un metodo *set(int i, int j, int sign)* la cui implementazione viene mostrata in Figura 1.

```
public void set(int i, int j, int sign) {
    if(i >= 0 && j >=0 && i < 3 && j < 3) {
        if(sign == 1) {
            board[i][j] = 1;
        }
        else {
            board[i][j] = 2;
        }
    }
}
```

Figure 1: Metodo *set*

Il metodo *set* controlla che i parametri i e j siano indici validi della griglia e, se corretti, aggiorna la cella a seconda del valore *sign*.

Metodo *won* La classe deve disporre di un metodo booleano *won(int sign)* che dica se il segno *sign* ha vinto.

JML

Scrivere in JML la seguente postcondizione al costruttore:

- tutte le celle della griglia valgono 0. [**pt. 1**]

Scrivere in JML le seguenti precondizioni al metodo *set(int i, int j, int sign)*:

- la cella identificata da (i, j) è vuota; [**pt. 0.5**]
- il valore *sign* indica correttamente uno dei due segni; [**pt. 0.5**]

Scrivere in JML le seguenti postcondizioni al metodo *set(int i, int j, int sign)*:

- la cella identificata da (i, j) viene aggiornata correttamente a *sign*; [**pt. 0.25**]
- la differenza in valore assoluto tra il numero di celle segnate con **X** e il numero di celle segnate con **O** è al massimo uno. [**pt. 2.25**]

Copertura

In JUnit, per il metodo *set*, scrivere:

- un test set che soddisfi la copertura delle decisioni, ma che non soddisfi anche la copertura delle condizioni; [**pt. 2**]
- un test set che soddisfi la copertura delle condizioni. [**pt. 2**]

Produttore - consumatore

Java [pt. 2]

Scrivere una classe *ProdCons* che simuli il problema del produttore - consumatore. La classe deve disporre di un buffer che contenga gli elementi prodotti dal produttore. Il buffer deve essere modellato con un array di interi: una cella vale 0 se non contiene il prodotto, 1 se lo contiene. Un opportuno indice deve indicare la prima cella senza prodotti.

La classe deve disporre di un metodo *produce* per produrre un prodotto ed un metodo *consume* per consumare un prodotto.

Il costruttore della classe deve inizializzare il buffer alla dimensione comunicata tramite parametro. All'inizio il buffer non contiene prodotti.

Metodo *produce* Il metodo *produce* aggiunge un prodotto al buffer solo se c'è spazio. Deve ritornare un booleano: true o false a seconda che sia stato inserito o meno il prodotto.

Metodo *consume* Il metodo *consume* elimina un prodotto dal buffer se questo contiene almeno un prodotto. Deve ritornare un booleano: true o false a seconda che sia stato eliminato o meno un prodotto.

JML

Scrivere in JML la seguente precondizione al costruttore:

- la capacità con cui deve essere inizializzato il buffer deve essere compresa tra 1 e 100; [**pt. 0.5**]

Scrivere in JML le seguenti postcondizioni al metodo *produce*:

- il buffer contiene solo 0 o 1; [**pt. 1**]
- nel buffer non c'è mai un 1 dopo uno 0; [**pt. 1.25**]
- il numero di 1 nel buffer dopo l'esecuzione del metodo è uguale o al massimo maggiore di un'unità al numero di 1 presenti nel buffer prima dell'esecuzione del metodo. [**pt. 1.75**]

Junit

In JUnit, scrivere i seguenti casi di test nella classe *ProdConsTest*.

- Per il metodo *produce*, scrivere:
  - un caso di test che mostri che, se il buffer non è pieno, l'inserimento di un prodotto viene eseguito (basandosi sul valore ritornato dal metodo); [**pt. 1**]
  - un caso di test che mostri che, se il buffer è pieno, l'inserimento di un prodotto non viene eseguito (basandosi sul valore ritornato dal metodo); [**pt. 1**]
- scrivere un caso di test che mostri che, appena dopo essere stato creato, il buffer non contiene prodotti (rendere il buffer pubblico per poterlo testare). [**pt. 1**]

Copertura

Scrivere in JUnit una test suite che soddisfi la copertura delle condizioni per il metodo *consume*; mettere i casi di test nella classe *ConsumeCoperturaCondizioni* e commentare in modo opportuno i singoli casi di test. **[pt. 1]**

Filosofi a cena

Java [pt. 2]

Mille filosofi siedono ad una tavola rotonda con un piatto di spaghetti davanti, una forchetta a destra ed una forchetta a sinistra: la forchetta destra di un filosofo corrisponde alla forchetta sinistra del filosofo alla sua destra.

La vita di un filosofo consiste in periodi in cui mangia alternati a periodi in cui pensa.

Ogni filosofo ha bisogno di due forchette per mangiare; può mangiare se la forchetta alla sua sinistra e quella alla sua destra sono libere.

Se un filosofo sta pensando non mangia e se mangia non pensa.

Scrivere una classe *DiningPhilosophers* che permetta di modellare il problema dei filosofi a cena.

All'inizio tutti i filosofi stanno pensando e le forchette sono tutte libere.

Metodo *startEating* La classe deve disporre di un metodo *startEating(int philo)* la cui implementazione viene mostrata in Figura 2.

```
public void startEating(int philo) {
    if(!forkUsed[philo] && !forkUsed[(philo + 1)%1000]) {
        eating[philo] = true;
        forkUsed[philo] = true;
        forkUsed[(philo + 1)%1000] = true;
    }
}
```

Figure 2: Metodo *startEating*

Il metodo *startEating* controlla se la forchetta sinistra e la forchetta destra del filosofo *philo* sono libere: nel caso le occupa e permette al filosofo di cominciare a mangiare.

Metodo *startThinking* La classe deve disporre di un metodo *startThinking(int philo)* che permetta al filosofo *philo* di cominciare a pensare. Nel caso in cui il filosofo stia già pensando lo stato della sua forchetta sinistra e destra non deve essere modificato.

Linee guida per la scrittura del codice

- Non è consentito usare campi e/o metodi statici;
- non è possibile aggiungere altri metodi rispetto a quelli descritti in precedenza.

JML

Scrivere in JML la seguente postcondizione al costruttore:

- tutti i filosofi pensano e tutte le forchette sono libere. [**pt. 1**]

Scrivere in JML la seguente precondizione al metodo *startEating(int philo)*:

- l'indice *philo* è un indice valido, cioè indica uno dei mille filosofi; [**pt. 0.25**]

Scrivere in JML le seguenti postcondizioni al metodo *startEating(int philo)*:

- non esistono due filosofi vicini che stanno mangiando; [**pt. 2.25**]
- almeno una tra la forchetta sinistra e la forchetta destra del filosofo *philo* è occupata. [**pt. 1**]

Copertura

In JUnit, per il metodo *startEating*, scrivere:

- un test set che soddisfi la copertura delle decisioni, ma che non soddisfi anche la copertura delle condizioni; [**pt. 2**]
- un test set che soddisfi la copertura delle condizioni. [**pt. 2**]

Scrivere una classe JUnit per ogni test set che soddisfa un criterio di copertura.
Per ogni test case scrivere un commento nel codice che indichi cosa si vuole coprire con quel test case.

Quadrato magico

Java [pt. 2.5]

Scrivere una classe *QuadratoMagico3* che rappresenti un quadrato 3×3; tale classe deve permettere di riempire il quadrato con dei valori e verificare se il quadrato è magico. All'inizio il quadrato deve essere vuoto (con tutte le celle a 0).

Metodo *set* La classe deve disporre di un metodo *set(int i, int j, int value)* che setta la cella identificata dagli indici (i, j) a *value*. Il metodo non deve eseguire alcun controllo sul fatto che gli indici siano corretti (compresi tra 0 e 2) e sul fatto che il valore sia corretto (compreso tra 1 e 9).

Metodo *isMagic* La classe deve disporre di un metodo booleano *isMagic()* che dice se il quadrato è magico. Un quadrato 3×3 è magico se tutte le righe , tutte le colonne e le due diagonali sommano a 15[2].

JML

Scrivere in JML le seguenti precondizioni al metodo *set*:

- l'indice i e l'indice j sono indici validi e la cella identificata da (i, j) è vuota; **[pt. 1.5]**
- il valore *value* è un valore valido; **[pt. 0.5]**
- il valore *value* non è già presente nel quadrato. **[pt. 2]**

Junit

2	7	6
9	5	1
4	3	8

Table 8: Esempio di quadrato magico 3×3

- In JUnit, per il metodo *set*, scrivere:
  - un caso di test in cui si mostri che la modifica di una cella vuota viene eseguita correttamente; **[pt. 1]**

[2]Un quadrato magico di ordine n contenente tutti gli interi da 1 a n^2 è detto *perfetto* o *normale*. La costante magica di questi quadrati è data dalla formula:

$$M(N) = \frac{1}{n}\sum_{k=1}^{n^2} k = \frac{1}{2}n(n^2+1)$$

  - un caso di test in cui si mostri che la modifica di una cella piena viene eseguita in modo errato. Il caso di test si aspetta che, se si prova a modificare una cella piena, la cella non dovrebbe essere modificata: invece il metodo la modifica. [**pt. 1**]
- scrivere in JUnit un caso di test che evidenzi che è possibile costruire un quadrato che sia magico; nella Tabella 8 viene mostrato un quadrato magico. [**pt. 1**]
- scrivere in JUnit un caso di test che evidenzi che è possibile costruire un quadrato che non sia magico. [**pt. 1**]

Rompicapo delle otto regine semplificato

Java [pt. 2.5]

Il *problema delle otto regine semplificato*[3] consiste nel posizionare otto regine su una scacchiera 8×8 in modo tale che nessuna di esse possa catturare un'altra regina muovendosi in orizzontale o in verticale (in questo problema la regina non può muoversi in diagonale). Quindi, una soluzione prevede che nessuna delle otto regine abbia la colonna o la riga in comune con un'altra regina.

Scrivere una classe *OttoRegineSimpl* che rappresenti una scacchiera 8×8 di booleani: la scacchiera deve memorizzare se in una cella è presente una regina. La classe deve disporre di un metodo che permetta di posizionare una regina sulla scacchiera.

All'inizio la scacchiera non deve contenere regine.

Metodo *place* La classe deve disporre di un metodo *place(int i, int j)* che permetta di posizionare in (i, j) una regina. Non bisogna fare alcun controllo sul fatto che i, j siano indici validi e che la regina non possa essere catturata.

Metodo *isSolved* La classe deve disporre di un metodo booleano *isSolved()* che dica se il rompicapo è stato risolto, cioè se sono state posizionate otto regine sulla scacchiera ed il posizionamento è tale per cui le regine non possono catturarsi muovendosi in orizzontale o in verticale (come in Tabella 9).

	a	b	c	d	e	f	g	h
8	♕							
7			♕					
6						♕		
5							♕	
4				♕				
3					♕			
2								♕
1		♕						

Table 9: Possibile soluzione del rompicapo.

[3] Versione semplificata del *rompicapo delle otto regine*: `http://en.wikipedia.org/wiki/Eight_queens_puzzle`

JML

Scrivere in JML le seguenti precondizioni al metodo *place(int i, int j)*:

- gli indici i, j sono indici validi della scacchiera; [**pt. 0.5**]
- la riga identificata da i non contiene nessuna regina; [**pt. 0.75**]
- la colonna identificata da j non contiene nessuna regina; [**pt. 0.75**]

Scrivere in JML la seguente postcondizione al metodo *place(int i, int j)*:

- il numero di regine sulla scacchiera è minore od uguale ad 8. [**pt. 2**]

JUnit

- In JUnit, per il metodo *place(int i, int j)*, scrivere:
  - un caso di test in cui si mostri che il posizionamento di due regine, la prima in (i, j) e la seconda in (h, k) (con (i, j) ed (h, k) scelti da voi in modo che le regine non possano catturarsi), avviene in modo corretto; [**pt. 1**]
  - un caso di test in cui si mostri che il posizionamento di una regina in una cella in cui può essere catturata viene eseguito portando la scacchiera in una configurazione errata. Il caso di test si aspetta che, se si prova a posizionare una regina in una cella in cui può essere catturata, il posizionamento non dovrebbe essere eseguito: invece il metodo esegue il posizionamento. [**pt. 1**]
- scrivere in JUnit un caso di test che evidenzi che la scacchiera impostata dal costruttore non contiene la soluzione del rompicapo; [**pt. 1**]
- scrivere in JUnit un caso di test che evidenzi che è possibile eseguire una serie di 8 posizionamenti che portano alla soluzione del rompicapo; nella tabella 9 potete trovare un possibile soluzione del rompicapo e, quindi, ricavare i parametri attuali delle otto chiamate del metodo *place(int i, int j)*. [**pt. 1**]

Forza quattro

Java [pt. 2]

Forza quattro è un gioco da tavolo in cui due giocatori (*rosso* e *nero*) devono riuscire a creare una sequenza di 4 delle loro pedine (in verticale o in orizzontale, non in diagonale) su una scacchiera posta in verticale.

Poiché la scacchiera è posta in verticale, una pedina inserita in una certa colonna va sempre a occupare la posizione libera situata più in basso nella colonna stessa.

X			
X			
X		**O**	
X	**O**	**O**	

Table 10: Esempio di scacchiera in cui ha vinto il rosso (**X**).

Scrivere una classe *ForzaQuattro* che rappresenti una scacchiera 4×4 di interi; un cella vale:

- 0 se non contiene alcuna pedina;
- 1 se contiene una pedina rossa;
- 2 se contiene una pedina nera.

All'inizio la scacchiera è vuota.

Metodo *addPedina* La classe deve disporre di un metodo *addPedina(int colore, int colonna)* la cui implementazione viene mostrata in Figura 3.

```
public void addPedina(int color, int column) {
    if(color > 0 && color < 3 && column >= 0 && column < 4) {
        if(freeRow[column] < 4) {
            board[freeRow[column]][column] = color;
            freeRow[column]++;
        }
    }
}
```

Figure 3: Metodo *addPedina*

Il metodo *addPedina* controlla che il parametro *color* indichi uno dei due colori (1 o 2) e che *column* indichi una colonna della scacchiera: se i parametri sono corretti, viene eseguito l'inserimento sole se c'è spazio nella colonna selezionata.

Metodo *won* La classe deve disporre di un metodo booleano *won(int color)* che dica se il colore *color* ha vinto.

JML

Scrivere in JML la seguente postcondizione al costruttore:

- tutte le celle della scacchiera valgono 0. [**pt. 1**]

Scrivere in JML le seguenti precondizioni al metodo *addPedina(int color, int column)*:

- il parametro *color* indica un colore valido; [**pt. 0.25**]
- il parametro *column* indica una colonna valida; [**pt. 0.25**]

Scrivere in JML le seguenti postcondizioni al metodo *addPedina(int color, int column)*:

- il contenuto delle colonne diverse da *column* dopo l'esecuzione del metodo è uguale al contenuto prima dell'esecuzione del metodo; [**pt. 1.5**]
- la differenza in valore assoluto tra il numero di celle rosse e il numero di celle nere è al massimo uno. [**pt. 1.5**]

Copertura

In JUnit, per il metodo *addPedina*, scrivere:

- un test set che soddisfi la copertura delle decisioni, ma che non soddisfi anche la copertura delle condizioni; [**pt. 2**]
- un test set che soddisfi la copertura delle condizioni. [**pt. 2**]

NaturalSet

Java [pt. 2]

Scrivere una classe *NaturalSet* che rappresenti un insieme di numeri naturali. La classe rappresenta l'insieme usando un array di interi: le celle libere dell'array contengono il valore -1.

Il costruttore della classe deve inizializzare l'array usando come dimensione il valore che viene passato come parametro al costruttore. All'inizio l'insieme è vuoto.

Metodo *remove* La classe deve disporre di un metodo *remove(int value)* che permetta di rimuovere un elemento dall'insieme; il metodo è mostrato nel Codice 1. Se il valore *value* è presente nell'insieme, il metodo lo rimuove e lo ritorna, altrimenti viene ritornato il valore -1.

```
public int remove(int value) {
    int i = 0;
    while(i < set.length) {
        if(set[i] == value) {
            set[i] = -1;
            return value;
        }
        i++;
    }
    return -1;
}
```

Codice 1: Metodo *remove*

Metodo *add* La classe deve disporre di un metodo booleano *add(int value)* che permetta di aggiungere un elemento all'insieme. Se il valore *value* non è già presente nell'insieme, deve essere inserito e il metodo deve ritornare *true*; altrimenti, se il valore è già presente nell'insieme, non deve essere inserito e il metodo deve ritornare *false*[4].

Il metodo, invece, non deve controllare che il valore che viene inserito sia positivo.

JML

Scrivere in JML la seguente precondizione al costruttore:

- la dimensione massima dell'insieme deve essere compresa tra 1 e 100. **[pt. 0.5]**

Scrivere in JML la seguente postcondizione al costruttore:

- l'insieme non contiene elementi. **[pt. 0.5]**

Scrivere in JML la seguente precondizione al metodo *add(int value)*:

[4]Per l'implementazione bisogna notare che non è possibile sapere a priori quale cella è libera, perché, a causa dell'esecuzione del metodo *remove*, l'array potrebbe alternare celle occupate (con valore maggiore od uguale a zero) a celle libere (con valore uguale a -1).

- il valore *value* è positivo. [**pt. 0.5**]

Scrivere in JML le seguenti postcondizioni al metodo *add(int value)*:

- nell'insieme non esistono due elementi uguali; [**pt. 1.5**]
- il numero di elementi dell'insieme dopo l'esecuzione del metodo è uguale o al massimo maggiore di un'unità al numero di elementi dell'insieme prima dell'esecuzione del metodo. [**pt. 1.5**]

Junit

In JUnit, scrivere per il metodo *add* nella classe *NaturalSetTest*:

- un caso di test che mostri che, se si prova ad aggiungere due volte lo stesso elemento, la seconda volta l'elemento non viene aggiunto (basandosi sul valore ritornato dal metodo); [**pt. 1**]
- un caso di test che mostri che, se si prova ad aggiungere un numero negativo, il numero viene aggiunto (basandosi sul valore ritornato dal metodo); il caso di test si aspetta che il codice sia corretto e quindi che, se si aggiunge un numero negativo, il numero non dovrebbe essere aggiunto: il metodo, invece, aggiunge l'elemento; [**pt. 1**]
- un caso di test che mostri che, in un insieme inizializzato con 10 posizioni libere, si possono inserire 10 elementi distinti. [**pt. 1**]

Copertura

Scrivere in JUnit una test suite che soddisfi la copertura delle decisioni per il metodo *remove*; mettere i test case nella classe *RemoveCoperturaDecisioni* e commentare in modo opportuno i singoli test case. [**pt. 1**]

Tombola

Java [pt. 2]

Scrivere una classe *Tombola* che modelli le azioni di un croupier nel gioco della tombola. La classe deve modellare con un array unidimensionale di interi (*tabellone*) il tabellone del croupier (il tabellone contiene 90 numeri). Il croupier può aggiungere delle pedine sui numeri; nell'array, il valore di ogni cella di indice i indica il numero di pedine posizionate sul numero $i + 1$.

Il costruttore della classe deve inizializzare il tabellone, facendo in modo che tutte le celle non contengano pedine.

Metodo *addPedina* La classe deve disporre di un metodo *addPedina(int numero)* che permetta di aggiungere una pedina sul tabellone. Il metodo è mostrato nel Codice 2.

```
public void addPedina(int numero) {
    if(numero >= 1 && numero <= 90 && tabellone[numero - 1] == 0) {
        tabellone[numero - 1] = tabellone[numero - 1] + 1;
    }
}
```

Codice 2: Metodo *addPedina(int numero)*

Il metodo controlla che il numero inserito sia un numero corretto (compreso tra 1 e 90) e che la cella non contenga già una pedina.

Metodo *isCorrect* La classe deve anche disporre di un metodo booleano *isCorrect(int[] numeroCartella, boolean[] numeroPresente)* che prenda in input un array di interi *numeroCartella*, che contiene i numeri stampati su una cartella, e un array di booleani *numeroPresente*, che riporta quali numeri sono stati selezionati dal giocatore sulla cartella. Per esempio, questi due array

numeroCartella	**numeroPresente**
{1, 12, 45, 50, 72}	{false, true, true, false, false}

rappresentano un cartella con stampati i numeri {1, 12, 45, 50, 72}, su cui il giocatore ha marcato i numeri 12 e 45.

Il metodo deve controllare che l'utente abbia marcato correttamente i numeri usciti sul tabellone, cioè che abbia segnato quelli che sono usciti e che non abbia segnato quelli che non sono usciti. Se la cartella è stata marcata in modo corretto il metodo ritorna *true*, altrimenti ritorna *false*.

JML

Scrivere in JML le seguenti postcondizioni al metodo *addPedina(int numero)*:

- non esiste una cella che contiene più di una pedina; **[pt. 1.25]**
- il prodotto di tutte le celle è minore o uguale a 1. **[pt. 1.25]**

Scrivere in JML le seguente precondizioni al metodo *isCorrect(int[] numeroCartella, boolean[] numeroPresente)*:

- l'array *numeroCartella* e l'array *numeroPresente* sono entrambi lunghi 5 posizioni; [**pt. 0.5**]
- tutti i numeri contenuti in *numeroCartella* sono compresi nell'intervallo [1,90]. [**pt. 1.5**]

Junit

In JUnit, scrivere i seguenti casi di test nella classe *TombolaTest*.

- Per il metodo *addPedina*, scrivere un caso di test che mostri che tutti i numeri compresi nell'intervallo [1,90] possono essere aggiunti al tabellone (rendere pubblico il campo *tabellone* per facilitare la scrittura del test); [**pt. 1**]
- Scrivere un caso di test che mostri che il metodo *isCorrect* è in grado di identificare un giocatore che non bara, cioè che ha segnato correttamente i numeri che sono usciti sul tabellone. [**pt. 1**]
- Scrivere un caso di test che mostri che il metodo *isCorrect* è in grado di scoprire un giocatore che bara, cioè che ha segnato dei numeri che non sono usciti sul tabellone. [**pt. 1**]

Copertura

Scrivere in JUnit una test suite per il metodo *addPedina* che soddisfi la copertura delle condizioni; mettere i casi di test nella classe *AddPedinaCoperturaCondizioni* e commentare in modo opportuno i singoli casi di test. [**pt. 1**]

Sudoku

Java [pt. 2.5]

Scrivere una classe *Sudoku* che permetta di giocare ad una versione semplificata del sudoku in cui la scacchiera è di dimensione 4×4. Una cella si considera vuota se contiene il numero 0. Il gioco è risolto se:

- ogni riga contiene tutti i numeri dell'intervallo $[1, 4]$ senza ripetizioni;
- ogni colonna contiene tutti i numeri dell'intervallo $[1, 4]$ senza ripetizioni;
- le quattro sotto-scacchiere 2×2 contengono tutti i numeri dell'intervallo $[1, 4]$ senza ripetizioni.

La Tabella 11 mostra un esempio di sudoku non risolto, mentre la Tabella 12 mostra un esempio di sudoku risolto.

1	2	0	0
0	0	1	0
2	0	4	0
0	0	0	3

Table 11: Sudoku non risolto

1	2	3	4
3	4	1	2
2	3	4	1
4	1	2	3

Table 12: Sudoku risolto

Il costruttore della classe deve inizializzare la scacchiera in modo che il sudoku sia vuoto.

Metodo *add* La classe deve disporre di un metodo booleano *add(int r, int c, int value)* che permetta di aggiungere un numero nella scacchiera; il metodo ritorna *true* se l'inserimento viene eseguito, *false* altrimenti. Il metodo è mostrato nel Codice 3.

```
public boolean add(int r, int c, int value) {
    if(r >= 0 && r <= 3 && c >= 0 && c <= 3 && value >= 1 && value <= 4) {
        board[r][c] = value;
        return true;
    }
    else
        return false;
}
```

Codice 3: Metodo *add(int r, int c, int value)*

Metodo *isMaybeSolved* La classe deve anche disporre di un metodo booleano *isMaybeSolved* che dica se la scacchiera soddisfa una condizione necessaria (ma non sufficiente) affinché possa essere considerata risolta:

- ogni riga deve sommare a 10;
- ogni colonna deve sommare a 10;
- le quattro sotto-scacchiere 2×2 devono sommare tutte a 10.

JML

Scrivere in JML la seguente precondizione al metodo *add(int r, int c, int value)*:

- tutti gli elementi sulla riga *r* sono diversi da *value*. [**pt. 1**]

Scrivere in JML le seguenti postcondizioni al metodo *add(int r, int c, int value)*:

- il numero di celle vuote è minore od uguale a 16; [**pt. 1.5**]
- il metodo ha ritornato *true* se e solo se la decisione della regola **if** contenuta nel metodo valuta a *true*. [**pt. 1.5**]

Junit

In JUnit, scrivere i seguenti casi di test nella classe *SudokuTest*.

- Per il metodo *add*, scrivere:
  - un caso di test che mostri che, se i parametri attuali sono corretti, il valore viene aggiunto alla scacchiera (basandosi sul valore ritornato dal metodo); [**pt. 1**]
  - un caso di test in cui si mostri che si può aggiungere un numero alla scacchiera che non soddisfa le regole del sudoku (ad esempio un numero già contenuto nella riga). Il caso di test si aspetta che, se si prova a mettere un numero in una riga che contiene già quel numero in una posizione diversa, l'aggiunta non dovrebbe essere eseguita: invece il metodo esegue l'aggiunta. [**pt. 1**]
- Scrivere un caso di test che mostri che, se si inseriscono i numeri come mostrato dalla tabella 12, il metodo *isMaybeSolved* dice che il sodoku potrebbe contenere la soluzione. [**pt. 1**]

Copertura

Scrivere in JUnit una test suite che soddisfi la copertura delle condizioni per il metodo *add*; mettere i test case nella classe *AddCoperturaCondizioni* e commentare in modo opportuno i singoli test case. [**pt. 1**]

Ostello

Java [pt. 2]

Scrivere una classe *Ostello* per modellare un ostello. L'ostello ha 50 letti in totale, distribuiti in 10 stanze da 5 letti ciascuna: ogni letto può essere libero o occupato.

Lo stato dei letti nelle stanze deve essere modellato con una matrice di booleani (le righe sono le stanze e le colonne i letti).

All'inizio tutti i letti sono liberi.

Metodo *checkin* La classe dispone di un metodo `checkin(int stanza, int letto)`, mostrato in Fig. 4, che permette di occupare un determinato `letto` in una determinata `stanza`.

```
public boolean checkin(int stanza, int letto) {
    if(stanza >= 0 & stanza < 10 & letto >= 0 & letto < 5) {
        if(lettoLibero[stanza][letto]) {
            lettoLibero[stanza][letto] = false;
            return true;
        }
    }
    return false;
}
```

Figure 4: Metodo *checkin*

Il metodo controlla che i valori `letto` e `stanza` siano corretti e che il letto selezionato sia libero. Il metodo ritorna *true* se il letto viene occupato, *false* altrimenti.

Metodo *ottimizzaOstello* L'ostello vuole minimizzare il numero di stanze usate. A tal fine la classe deve disporre di un metodo void `ottimizzaOstello()` che sposti tutti i clienti nelle prime stanze libere (a partire dalla prima stanza e dal primo letto). Per esempio, se c'è un solo cliente per ogni stanza (quindi in totale ci sono 10 clienti), il metodo `ottimizzaOstello()` sposta tutti i clienti nelle prime due stanze.

JML

Scrivere in JML la seguente postcondizione al costruttore:

- all'inizio tutti i letti sono liberi. [**pt. 1**]

Scrivere in JML le seguenti postcondizioni al metodo `checkin(int stanza, int letto)`:

- tutte i letti diversi dal letto indentificato dai parametri `stanza` e `letto` non hanno cambiato il loro stato; [**pt. 1**]
- il numero di letti occupati è minore o uguale a 50; [**pt. 1**]
- se `stanza` è negativo, il metodo ha ritornato *false*. [**pt. 1**]

Junit

In JUnit, scrivere:

- un caso di test che mostri che è possibile trovare un letto libero ed occuparlo (basandosi sul valore ritornato dal metodo); [**pt. 1**]
- un caso di test che mostri che non è possibile occupare un letto già occupato (basandosi sul valore ritornato dal metodo); [**pt. 1**]
- un caso di test che mostri che se si mettono 5 persone nell'ultima stanza e poi si ottimizza l'occupazione delle stanze dell'ostello, gli ospiti sono spostati tutti nella prima stanza; rendere pubblico il campo *lettoLibero* per facilitare la scrittura del test. [**pt. 1**]

Copertura

Scrivere in JUnit una test suite per il metodo `checkin` che soddisfi la copertura delle decisioni ma che non soddisfi anche la copertura delle condizioni. Creare una classe `CheckinCoperturaDecisioni` e commentare in modo opportuno i singoli casi di test. [**pt. 1.5**]

Queue

Java [pt. 2]

Scrivere una classe *Queue* che rappresenti un coda FIFO contenente stringhe. La classe rappresenta la coda usando un array di stringhe: le celle libere dell'array contengono la stringa vuota "".

Il costruttore della classe deve inizializzare l'array usando come dimensione il valore che viene passato come parametro al costruttore. All'inizio la coda è vuota.

Metodi *add* e *pop* La classe deve disporre di un metodo booleano *add(String str)* che permetta di aggiungere un elemento in fondo alla coda; il metodo è mostrato nel Codice 4. Se l'inserimento è possibile, il metodo aggiunge la stringa e ritorna *true*, altrimenti ritorna *false*.

La classe deve inoltre disporre di un metodo booleano *pop()* che permetta di rimuovere un elemento dalla cima della coda; il metodo è mostrato nel Codice 5. Se la coda non è vuota ritorna la stringa in cima, altrimenti ritorna *null*.

```
public boolean add(String str) {
    if(size < capacity) {
        arr[(top + size)%capacity] = str;
        size++;
        return true;
    }
    return false;
}
```

Codice 4: Metodo *add*

```
public String pop() {
    if(size > 0) {
        String str = arr[top];
        arr[top] = "";
        top = (top + 1)%capacity;
        size--;
        return str;
    }
    else {
        return null;
    }
}
```

Codice 5: Metodo *pop*

Metodo *numberOfDistinctWords* La classe deve disporre di un metodo *numberOfDistinctWords* che ritorni il numero di parole distinte presenti nella coda (quindi non bisogna considerare le celle vuote (stringa "") dell'array). Tabella 13 mostra alcuni esempi.

array	**# parole distinte**
"pippo", "", "pippo", "sdsada"	2
"pippo", "", "pluto", ""	2
"", "", "", ""	0
"pippo", "dsad", "pluto", "wwds"	4

Table 13: Conteggio parole distinte

JML

Scrivere in JML la seguente precondizione al costruttore:

- la dimensione massima della coda deve essere compresa tra 2 e 100. [**pt. 0.25**]

Scrivere in JML la seguente postcondizione al costruttore:

- la coda non contiene elementi. [**pt. 1**]

Scrivere in JML la seguente precondizione al metodo *add(String str)*:

- la lunghezza della stringa che si vuole aggiungere è compresa tra 2 e 10 caratteri. [**pt. 0.25**]

Scrivere in JML le seguenti postcondizioni al metodo *add(String str)*:

- il metodo ha ritornato *true* se e solo se, prima dell'esecuzione del metodo, la dimensione della coda era minore della sua capacità massima; [**pt. 1.5**]
- la media delle lunghezze delle stringhe contenute nell'array (tutte le stringhe, anche quelle vuote) è minore od uguale a 10. [**pt. 1.5**]

Junit

In JUnit, scrivere:

- un caso di test che mostri che, se si prova ad prelevare un elemento dalla coda vuota, viene ritornato *null*; [**pt. 1**]
- un caso di test che mostri che, se si prova ad aggiungere una stringa alla coda piena, la stringa non viene aggiunta (basandosi sul valore ritornato dal metodo); [**pt. 1**]
- un caso di test che mostri che il funzionamento della coda è effettivamente FIFO; [**pt. 1**]
- un caso di test che testi il funzionamento del metodo *numberOfDistinctWords*. [**pt. 1**]

Fila

Java [pt. 2]

Scrivere una classe *Fila* per modellare una fila ad uno sportello. Ogni utente ritira un numero da 0 a 199 per effettuare un'operazione allo sportello. Ogni operazione corrisponde a un numero stimato di minuti (da 1 a 15). Gli operatori allo sportello chiamano il prossimo numero, in ordine crescente a partire da 0 fino a 199. Il primo utente della giornata prende il numero 0. Ogni nuovo utente prende il numero successivo all'ultimo ritirato. Al massimo sono serviti 200 utenti.

La fila deve essere rappresentata da un array (`fila[]`) da 0 a 199 dove ogni cella di indice i corrisponde al numero di minuti stimati (intero da 1 a 15) per l'operazione dell'utente che ha preso il numero i. Due variabili, che vanno da 0 a 199, contengono il numero del primo posto della fila libero (`prossimoInFila`) e il prossimo utente che sarà servito (`prossimoServito`). Per esempio, se la fila è composta da cinque utenti (da 0 a 4) e l'utente 1 è allo sportello si ha che `prossimoServito` è 2 e `prossimoInFila` è 5.

Il costruttore della classe deve creare e inizializzare una fila vuota (con minuti tutti a 0) e le due variabili `prossimoServito` e `prossimoInFila` in modo coerente.

Metodo *ritiroNumero* La classe dispone di un metodo `ritiroNumero(int minuti)` che restituisce il numero di fila all'utente e inserisce nella fila i `minuti` stimati per l'operazione. Nel caso siano già stati inseriti in fila 200 utenti il metodo restituisce -1. Il metodo è mostrato in Codice 6.

```
public int ritiroNumero(int minuti) {
    int numero = −1;
    if(prossimoInFila<200 && prossimoInFila>=0) {
        fila[prossimoInFila] = minuti;
        numero = prossimoInFila++; }
    return numero;
}
```

Codice 6: Metodo `ritiroNumero(int minuti)`

Metodo *chiamataAlloSportello* La classe deve disporre di un metodo `chiamataAlloSportello()` che ritorni un intero (ovvero il prossimo numero da servire e che quindi viene chiamato allo sportello) e che aggiorni di conseguenza `prossimoServito`. Nel caso in cui la fila sia finita (ovvero non ci sono utenti in attesa o stati stati serviti tutti e 200 gli utenti previsti) il metodo deve restituire -1.

Metodo *stimaMinuti* La classe deve disporre di un metodo `stimaMinuti(int numero)` che, dato un numero in fila, ritorni un intero che contiene la stima del numero di minuti che deve aspettare l'utente prima che tocchi a lui (senza considerata l'utente allo sportello). Per esempio se `prossimoServito` è 3 (ovvero 2 è attualmente allo sportello) e il `numero` dell'utente è 5 con una fila dove: i minuti dell'utente 2 sono stimati a 5, quelli di 3 sono 10, quelli di 4 sono 2, e quelli di 5 sono 3; il metodo deve restituire 12. Nel caso in cui `numero` corrisponde ad un utente non in fila (ovvero un numero già chiamato o che non è stato ancora ritirato) il metodo deve restituire -1.

JML

Scrivere in JML il seguente invariante:

1. `prossimoInFila` ha un valore tra 0 e 200 inclusi e `prossimoServito` non è mai strettamente superiore a `prossimoInFila`. [**pt. 0.75**]

Scrivere in JML la seguente postcondizione al costruttore:

2. non ci sono utenti in fila, verificando esclusivamente il contenuto dell'array `fila[]`. [**pt. 0.50**]

Scrivere in JML la seguente precondizione al metodo `ritiroNumero(int minuti)`:

3. i minuti di attesa di un utente sono compresi tra 1 e 15 inclusi. [**pt. 0.25**]

Scrivere in JML le seguenti postcondizioni al metodo `ritiroNumero(int minuti)`:

4. tutti gli elementi dell'array `fila[]` di indice strettamente inferiore a `prossimoInFila` hanno valore strettamente maggiori di 0. [**pt. 0.75**]
5. almeno un utente è stato inserito in fila, verificando esclusivamente il contenuto dell'array `fila[]`. [**pt.1**]
6. nell'array `fila[]` non c'è mai uno valore a 0 prima di uno strettamente maggiore di 0. [**pt. 1.5**]

Scrivere in JML la seguente postcondizione al metodo `stimaMinuti(int numero)`:

7. se `numero` è già stato chiamato il metodo restituisce -1. [**pt. 0.75**]

Junit

In JUnit, scrivere i seguenti casi di test nella classe `FilaTest`.

1. Per il metodo `ritiroNumero(int minuti)`, scrivere un caso di test che mostri che, se si prova a ritirate un numero quando la fila è piena il numero non viene fornito (ovvero si ottiene -1); [**pt. 1**]
2. Scrivere un caso di test che mostri che è possibile riempire completamente la fila in modo che l'ultimo utente debba aspettare esattamente 1000 minuti. [**pt. 1**]

Copertura

Scrivere in JUnit una test suite per il metodo `ritiroNumero` che soddisfi la copertura delle *decisioni*. Creare una classe `FilaCoperturaDecisioni` e commentare in modo opportuno i singoli casi di test. (Si ricorda che il metodo ha successo quando restituisce un valore diverso da -1.) Per testare la copertura mettere la variabile `prossimoInFila` come public. [**pt. 1**]

Undirected graph

Java [pt. 2.5]

Scrivere una classe *UndirectedGraph* che rappresenti un grafo non orientato. La classe deve memorizzare gli archi usando una matrice booleana di incidenza di dimensione $n \times n$ (dove n è il numero di nodi): la cella (i, j) della matrice vale true se c'è un arco dal nodo i al nodo j (poiché l'arco non è orientato, anche (j, i) deve valere true).

Il costruttore della classe deve inizializzare la matrice di incidenza facendo in modo che questa rappresenti un grafo di n nodi senza archi: il valore n deve essere passato come parametro al costruttore.

Metodo *addArc* La classe deve disporre di un metodo *addArc(int nodeA, int nodeB)* che permetta di aggiungere archi al grafo; il metodo è mostrato nel Codice 7.

```
public boolean addArc(int nodeA, int nodeB) {
   if(nodeA >= 0 && nodeA < incidenceMatrix.length &&
         nodeB >= 0 && nodeB < incidenceMatrix.length) {
      incidenceMatrix[nodeA][nodeB] = true;
      incidenceMatrix[nodeB][nodeA] = true;
      return true;
   }
   else {
      return false;
   }
}
```

Codice 7: Metodo *addArc(int nodeA, int nodeB)*

Metodo *cycle3* La classe deve anche disporre di un metodo booleano *cycle3* che dica se la il grafo contiene almeno un ciclo di lunghezza 3. Figura 5 contiene un esempio di ciclo di lunghezza 3.

JML

Scrivere in JML la seguente precondizione al costruttore:

- il numero di nodi del grafo deve essere compreso tra 1 e 100. **[pt. 0.25]**

Scrivere in JML la seguente postcondizione al costruttore:

- il grafo non contiene archi. **[pt. 1]**

Scrivere in JML la seguente precondizione al metodo *addArc(int nodeA, int nodeB)*:

- non esiste un arco tra *nodeA* e *nodeB*. **[pt. 1]**

Scrivere in JML la seguente postcondizione al metodo *addArc(int nodeA, int nodeB)*:

- il numero di archi del grafo dopo l'esecuzione del metodo è uguale o al massimo maggiore di un'unità al numero di archi del grafo prima dell'esecuzione del metodo (attenzione che ogni arco viene rappresentato due volte). **[pt. 1.75]**

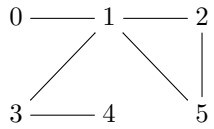

Figure 5: Grafo non orientato con ciclo di lunghezza 3 tra i nodi 1, 2 e 5

Junit

In JUnit, scrivere i seguenti casi di test nella classe *DirectedGraphTest*.

- Per il metodo *addArc*, scrivere:
  - un caso di test che mostri che, se gli indici sono corretti, l'arco viene aggiunto (basandosi sul valore ritornato dal metodo); **[pt. 1]**
  - un caso di test che mostri che, se gli indici non sono corretti, l'arco non viene aggiunto (basandosi sul valore ritornato dal metodo); **[pt. 1]**
- scrivere un caso di test che mostri che un determinato grafo da voi costruito contiene un ciclo di lunghezza 3. **[pt. 1]**

Copertura

Scrivere in JUnit una test suite che soddisfi la copertura delle condizioni per il metodo *addArc*; mettere i test case nella classe *AddArcCoperturaCondizioni* e commentare in modo opportuno i singoli test case. **[pt. 1]**

Ora di punta

Java [pt. 2]

Scrivere una classe *OraDiPunta* che modelli una versione semplificata del gioco *Ora di punta*. Su una griglia 6×6 sono disposte 6 macchine, numerate con i numeri da 1 a 6 (ogni macchina ha dimensione 1×1); celle vuote sono indicate con 0. Obiettivo del gioco è portare la macchina numero 1 (chiamata macchina *rossa*) all'*uscita* della griglia, che corrisponde alla cella con indici $(2, 5)$ (le coordinate iniziano da 0).

	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	0	0	0	2
2	0	0	1	0	0	**3**
3	0	0	0	0	0	4
4	0	5	0	0	0	0
5	0	0	6	0	0	0

Table 14: Configurazione della griglia del gioco *Ora di punta* (in grassetto viene mostrata l'uscita)

Tabella 14 mostra la configurazione iniziale del gioco; il numero in grassetto indica l'uscita. Il costruttore della classe deve disporre le macchine sulla griglia come mostrato in Tabella 14.

Metodo *spostaMacchina* La classe deve disporre di un metodo booleano *spostaMacchina(int riga, int colonna, int dir)* che permetta di muovere una macchina sulla griglia; il metodo ritorna *true* se una macchina è stata spostata, *false* altrimenti. I parametri *riga* e *colonna* indicano una cella della griglia; *dir* indica una direzione nel seguente modo:

- 1: verso l'alto;
- 2: verso destra;
- 3: verso il basso;
- 4: verso sinistra.

Il metodo dove controllare che i valori passati siano corretti: *riga* e *colonna* sono indici corretti della griglia e *dir* è una delle quattro direzioni. Se in *riga* e *colonna* non c'è una macchina, niente deve accadere. Se una macchina non si può spostare in direzione *dir* perché ostruita da un'altra macchina o dai limiti della griglia, non deve essere spostata.

Metodo *macchinaRossaUscita* La classe dispone di un metodo booleano *macchinaRossaUscita* che dice se la macchina rossa ha raggiunto l'uscita; il metodo è mostrato in Codice 8.

```
public boolean macchinaRossaUscita() {
    return griglia[2][5] == 1;
}
```

Codice 8: Metodo *macchinaRossaUscita()*

JML

Scrivere in JML la seguente postcondizione al costruttore:

- nella griglia ci sono 6 celle non vuote. [**pt. 1.25**]

Scrivere in JML la seguente precondizione al metodo *spostaMacchina(int riga, int colonna, int dir)*:

- i parametri attuali identificano una cella della griglia e una delle quattro direzioni. [**pt. 0.5**]

Scrivere in JML le seguenti postcondizioni al metodo *spostaMacchina(int riga, int colonna, int dir)*:

- il prodotto di tutte le celle è 0; [**pt. 1.25**]
- non esiste una cella con valore minore di 0. [**pt. 1.5**]

Junit

In JUnit, scrivere i seguenti casi di test nella classe *OraDiPuntaTest*.

- Per il metodo *spostaMacchina(int riga, int colonna, int dir)*, scrivere:
  - un caso di test che mostri che, se si prova a spostare una macchina in una cella libera, lo spostamento viene eseguito (basandosi sul valore ritornato dal metodo); [**pt. 1**]
  - un caso di test che mostri che, se si prova a spostare una macchina in una cella occupata, lo spostamento non viene eseguito (basandosi sul valore ritornato dal metodo); [**pt. 1**]
- Scrivere un caso di test che mostri che è possibile portare la macchina rossa all'uscita. [**pt. 1**]

Copertura

Scrivere in JUnit una test suite per il metodo *spostaMacchina* che soddisfi la copertura delle istruzioni (creare una classe *SpostaMacchinaCoperturaIstruzioni*). Commentare in modo opportuno i singoli casi di test. [**pt. 1**]

Rubik2D

Java [pt. 2]

Scrivere una classe `Rubik2D` per rappresentare una versione semplificata del cubo di Rubik in cui solo una faccia del cubo viene rappresentata.

All'inizio, la configurazione della faccia è quella mostrata in Fig. 6.

1	0	1
2	1	2
0	2	0

Figure 6: Configurazione iniziale

Metodo `gira` La classe dispone di un metodo booleano `gira` che, dati un indice di colonna `idCol`, sposta verso l'alto la colonna (in modo ciclico); il metodo è mostrato in Codice 9.

```
public boolean gira(int idCol) {
    if(idCol >= 0 && idCol < 3) {
        int primo = faccia[0][idCol];
        for(int i = 0; i < 2; i++) {
            faccia[i][idCol] = faccia[i + 1][idCol];
        }
        faccia[2][idCol] = primo;
        return true;
    }
    return false;
}
```

Codice 9: Metodo `gira(int idCol)`

Il metodo ritorna *true* se `idCol` indica una colonna valida e quindi lo spostamento viene fatto, *false* altrimenti.

Pe esempio, eseguendo il metodo `gira(2)` della configurazione iniziale, si ottiene la configurazione mostrata in Fig. 7.

1	0	2
2	1	0
0	2	1

Figure 7: Configurazione ottenibile eseguendo `gira(2)` a partire dalla configurazione iniziale

Metodo `isSolved` La classe deve disporre di un metodo booleano `isSolved()` che dica se la faccia è *risolta*, cioè se su ogni riga c'è lo stesso numero. Un esempio di faccia risolta viene mostrata in Fig. 8.

2	2	2
0	0	0
1	1	1

Figure 8: Esempio di faccia risolta

JML

Scrivere in JML la seguente postcondizione al costruttore:

- il valore di tutte le celle della faccia sono comprese tra 0 e 2. [**pt. 1**]

Scrivere in JML le seguenti postcondizioni al metodo `gira(int idCol)`:

- ci sono tre celle con valore uguale a 0; [**pt. 1**]
- la somma di tutte le celle è 9; [**pt. 1**]
- le celle che non appartengono alla colonna `idCol` non hanno cambiato valore rispetto a prima dell'esecuzione del metodo. [**pt. 1.5**]

Junit

In JUnit, scrivere i seguenti casi di test nella classe `Rubik2DTest`.

- Per il metodo `gira(int idCol)`, scrivere:
  - un caso di test che mostri che è possibile eseguire lo spostamento su una colonna (basandosi sul valore ritornato dal metodo; [**pt. 1**]
  - un caso di test che mostri che, se si seleziona una colonna inesistente, nessuno spostamento viene eseguito (basandosi sul valore ritornato dal metodo). [**pt. 1**]
- Scrivere un caso di test che mostri che è possibile risolvere la faccia. [**pt. 1**]

Copertura

Scrivere in JUnit una test suite per il metodo `gira` che soddisfi la copertura delle condizioni. Creare una classe `GiraCoperturaCondizioni` e commentare in modo opportuno i singoli casi di test. [**pt. 1**]